Calc AlignmentData

```
################################################################
# Name     : SC_ALMT
# Function : Calc AlignmentData
# Summary  : Alignment Offset Value Calculation(Scribe Remodeling)
# Input    : XB000001 Start BIT
#          : XB000002 Alignment Offset Enable/Disable
#          : XW000001 HeadNo
# Address  : AW00000  Offset Address
# Output   : YB000000 EndBit
# Called By: MPS167(Scribe Data Management)
################################################################
```

0 0/0

IF
'XB000001' == true
XB000001 == true;

1 1/4 NL 2

EXPRESSION
'DF00000' = 'Upper Right Alignement Position' - 'Upper Left Alignment Position'
DF00000 = ML01158 - ML01156; // L1/Mark the distance between the left and right
'DF00002' = 'ML15104' - 'ML15108'
DF00002 = ML15104 - ML15108; // ΔL1 = RY - LY
'DF00004' = 'DF00002' / 'DF00000'
DF00004 = DF00002 / DF00000; // ΔL1 / L1

[DF00000] .../0011
[DF00002] .../0010
[DF00004] .../0013

2 2/13 NL 2

ASIN	[FD]Src	[FD]Dest
	DF00004	DF00006
	---	---

[DF00006]
.../0016 .../0137 .../0146
.../0158 .../0167 .../0179
.../0188 .../0200 .../0209
.../0254 .../0263 .../0273
.../0282 .../0292 .../0301
.../0311 .../0320

3 3/16 NL 2

EXPRESSION
'ML04884' = 'DF00006' * 1000
ML04884 = DF00006 * 1000;

X 사선 보정량 연산

4 4/19 NL 2

EXPRESSION
'DF00008' = '[EQ]스크라이브 X축 사선 보정 Y방향 보정량'
DF00008 = ML26220; // 스크라이브 X축 사선 보정 Y방향 보정량
'DF00010' = '[EQ]스크라이브 X축 사선 보정 X방향 기준 거리'
DF00010 = ML26222; // 스크라이브 X축 사선 보정 X방향 기준 거리

[DF00008]
.../0029w .../0036 .../0045
.../0054 .../0063
[DF00010]
.../0031w .../0037 .../0046
.../0055 .../0064

5 5/23 NL 2

==	[WLFQD]SrcA	[WLFQD]SrcB
	00000	ML23612

EXPRESSION
'DF00008' = 0
DF00008 = 0; // 스크라이브 X축 사선 보정 Y방향 보정량
'DF00010' = 1
DF00010 = 1; // 스크라이브 X축 사선 보정 X방향 기준 거리

[DF00008]
.../0020w .../0036 .../0045
.../0054 .../0063
[DF00010]
.../0022w .../0037 .../0046
.../0055 .../0064

6 7/33 NL 2

Calculation

EXPRESSION
'DF00020' = ('AL00000' - 'Upper Left Alignment Position')*('DF00008'/'DF00010')
DF00020 = (AL00000 - ML01156)*(DF00008/DF00010); // Start Position UPCX
'DF00022' = ('AL00002' - 'Upper Left Alignment Position')*('DF00008'/'DF00010')
DF00022 = (AL00002 - ML01156)*(DF00008/DF00010); // Start Position LOCX
'DF00024' = ('AL00008' - 'Upper Left Alignment Position')*('DF00008'/'DF00010')
DF00024 = (AL00008 - ML01156)*(DF00008/DF00010); // End Position UPCX
'DF00026' = ('AL00010' - 'Upper Left Alignment Position')*('DF00008'/'DF00010')
DF00026 = (AL00010 - ML01156)*(DF00008/DF00010); // End Position LOCX

[DF00020] .../0271
[DF00022] .../0290
[DF00024] .../0309
[DF00026] .../0328

CX보정 계산

7 8/69 NL 2

==	[WLFQD]SrcA	[WLFQD]SrcB
	C_Control._ CW00001 Scribe Direction (0: Forward / 1: Reverse)	00000

EXPRESSION
'DL00100' = 'AL00000' - 'Upper Left Alignment Position' + '[EQ]Left Upper Camera X ~ Left Upper Head Cutter Distance'
DL00100 = AL00000 - ML01156 + ML26302; // Start PositionLUCX
'DL00102' = 'AL00002' - 'Upper Left Alignment Position' + '[EQ]Left Upper Camera X ~ Left Upper Head Cutter Distance'
DL00102 = AL00002 - ML01156 + ML26302; // Start PositionLOCX
'DL00104' = 'AL00008' - 'Upper Left Alignment Position' + '[EQ]Left Upper Camera X ~ Left Upper Head Cutter Distance'
DL00104 = AL00008 - ML01156 + ML26302; // End Position LUCX
'DL00106' = 'AL00010' - 'Upper Left Alignment Position' + '[EQ]Left Upper Camera X ~ Left Upper Head Cutter Distance'
DL00106 = AL00010 - ML01156 + ML26302; // End Position LOCX

[DL00102]
.../0101w .../0170 .../0238w
.../0285
[DL00104]
.../0104w .../0191 .../0241w
.../0304
[DL00106]
.../0107w .../0212 .../0244w
.../0323

8 10/91 NL 2

==	[WLFQD]SrcA	[WLFQD]SrcB
	C_Control._ CW00001 Scribe Direction (0: Forward / 1: Reverse)	00001

EXPRESSION
'DL00100' = 'AL00000' - 'Upper Left Alignment Position'
DL00100 = AL00000 - ML01156; // Start PositionLUCX
'DL00102' = 'AL00002' - 'Upper Left Alignment Position'
DL00102 = AL00002 - ML01156; // Start PositionLOCX
'DL00104' = 'AL00008' - 'Upper Left Alignment Position'
DL00104 = AL00008 - ML01156; // End Position LUCX
'DL00106' = 'AL00010' - 'Upper Left Alignment Position'
DL00106 = AL00010 - ML01156; // End Position LOCX

[DL00102]
.../0081w .../0170 .../0238w
.../0285
[DL00104]
.../0085w .../0191 .../0241w
.../0304
[DL00106]
.../0089w .../0212 .../0244w
.../0323

9 12/109 NL 2

EXPRESSION
'DL00200' = 'ML15218'
DL00200 = ML15218; // LUCS Start Position
'DL00202' = 'ML15220'
DL00202 = ML15220; /// LLCS Start Position
'DL00204' = 'ML15222'
DL00204 = ML15222; // LUCS End Position
'DL00206' = 'ML15224'
DL00206 = ML15224; // LLCS End Position

[DL00202]
.../0161 .../0276
[DL00204]
.../0182 .../0295
[DL00206]
.../0203 .../0314

10 13/117 NL 2

==	[WLFQD]SrcA	[WLFQD]SrcB
	C_Control._ CW00001 Scribe Direction (0: Forward / 1: Reverse)	00000

EXPRESSION
// ML23702 : Left Camera~LUCX Cutter DIS.
'DL00014' = 'Upper Left Alignment Position' - '[EQ]Left Upper Camera X ~ Left Upper Head Cutter Distance'
DL00014 = ML01156 - ML26302;

11 15/126 NL 2

==	[WLFQD]SrcA	[WLFQD]SrcB
	C_Control._ CW00001 Scribe Direction (0: Forward / 1: Reverse)	00001

EXPRESSION
// ML23702 : Left Camera~LUCX Cutter DIS.
'DL00014' = 'Upper Left Alignment Position'
DL00014 = ML01156;

PositionX

12 17/134 NL 2

EXPRESSION

```
//ΔL4 = COSθ * L2
//ΔL5 = SINθ * L3
//PositionX = Left Mark Position X + LX - ΔL5 + ΔL4
'DF00150' = 'DL00014' + 'ML15102' + ('DL00200' * sin('DF00006')) + ('DL00100' * cos('DF00006'))
DF00150 = DL00014 + ML15102 + (DL00200 * sin(DF00006)) + (DL00100 * cos(DF00006));
'DF00152' = 'DL00014' + 'ML15102' + ('DL00202' * sin('DF00006')) + ('DL00102' * cos('DF00006'))
DF00152 = DL00014 + ML15102 + (DL00202 * sin(DF00006)) + (DL00102 * cos(DF00006));
'DF00154' = 'DL00014' + 'ML15102' + ('DL00204' * sin('DF00006')) + ('DL00104' * cos('DF00006'))
DF00154 = DL00014 + ML15102 + (DL00204 * sin(DF00006)) + (DL00104 * cos(DF00006));
'DF00156' = 'DL00014' + 'ML15102' + ('DL00206' * sin('DF00006')) + ('DL00106' * cos('DF00006'))
DF00156 = DL00014 + ML15102 + (DL00206 * sin(DF00006)) + (DL00106 * cos(DF00006));
```

[DF00150] .../0218
[DF00152] .../0220
[DF00154] .../0222
[DF00156] .../0224

13 18/218 NL 2

EXPRESSION

```
// Upper Cutter X 개시
'ML15900' = 'DF00150'
ML15900 = DF00150;
// Lower Cutter X 개시
'ML15902' = 'DF00152'
ML15902 = DF00152;
// Upper Cutter X 종료
'ML15908' = 'DF00154'
ML15908 = DF00154;
// Lower Cutter X 종료
'ML15910' = 'DF00156'
ML15910 = DF00156;
```

CY보정 계산

14 19/226 NL 2

==	[WLFQD]SrcA	[WLFQD]SrcB
	C_Control._ CW00001 Scribe Direction (0: Forward / 1: Reverse)	00001

EXPRESSION

```
'DL00100' = 'AL00000' - 'Upper Left Alignment Position' + 'ML26304'
DL00100 = AL00000 - ML01156 + ML26304; // Start PositionLUCX
'DL00102' = 'AL00002' - 'Upper Left Alignment Position' + 'ML26304'
DL00102 = AL00002 - ML01156 + ML26304; // Start PositionLOCX
'DL00104' = 'AL00008' - 'Upper Left Alignment Position'
DL00104 = AL00008 - ML01156; //  End Position LUCX
'DL00106' = 'AL00010' - 'Upper Left Alignment Position'
DL00106 = AL00010 - ML01156; //  End Position LOCX
```

[DL00102] .../0081w .../0101w .../0170 .../0285
[DL00104] .../0085w .../0104w .../0191 .../0304
[DL00106] .../0089w .../0107w .../0212 .../0323

PositionY

15 21/246 NL 2

EXPRESSION

```
// PositionY
// Upper Chuck Y Start Position
'ML15904' = 'ML15206'
ML15904 = ML15206;
// Lower Chuck Y Start Position
'ML15906' = 'ML15208'
ML15906 = ML15208;
// Upper Chuck Y End Position
'ML15912' = 'ML15214'
ML15912 = ML15214;
// Lower Chuck Y End Position
'ML15914' = 'ML15216'
ML15914 = ML15216;

// Shift Position
'ML15916' = 'ML15104' + ('DL00200' * cos('DF00006')) - ('DL00100' * sin('DF00006')) + 'DF00020'
ML15916 = ML15104 + (DL00200 * cos(DF00006)) - (DL00100 * sin(DF00006)) + DF00020;
'ML15918' = 'ML15104' + ('DL00202' * cos('DF00006')) - ('DL00102' * sin('DF00006')) + 'DF00022'
ML15918 = ML15104 + (DL00202 * cos(DF00006)) - (DL00102 * sin(DF00006)) + DF00022;
'ML15920' = 'ML15104' + ('DL00204' * cos('DF00006')) - ('DL00104' * sin('DF00006')) + 'DF00024'
ML15920 = ML15104 + (DL00204 * cos(DF00006)) - (DL00104 * sin(DF00006)) + DF00024;
'ML15922' = 'ML15104' + ('DL00206' * cos('DF00006')) - ('DL00106' * sin('DF00006')) + 'DF00026'
ML15922 = ML15104 + (DL00206 * cos(DF00006)) - (DL00106 * sin(DF00006)) + DF00026;
```

16 22/330 ELSE

17 23/331 END_IF

18 24/332 OnCoil (Always ON) — YB000000 (EndBit)

19 26/334 END